Introduction to Applied Machine Learning

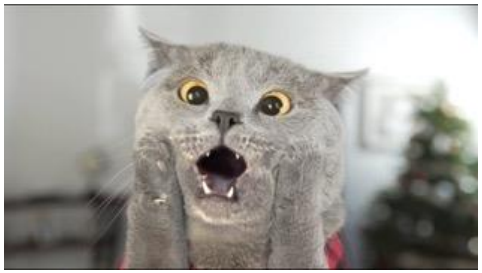
06: Convolutional Neural Networks and Image Recognition

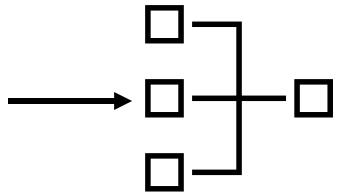
🎓 Machine Learning Adventure 🚀

1. 🤖 Introduction to Machine Learning
2. 🧠 Artificial Neural Networks and Computer Vision
3. 🔤 Transformers and Language Models
4. 🎨 GANs and Diffusion for Image Generation
5. 🎮 Reinforcement Learning and Behavior
6. 🏆 Final Projects

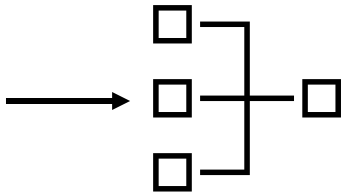
Image Recognition (Classification)

We want a classification model (typically as a conditional probability estimator) that takes an image as input and predicts a categorical label. For examples…





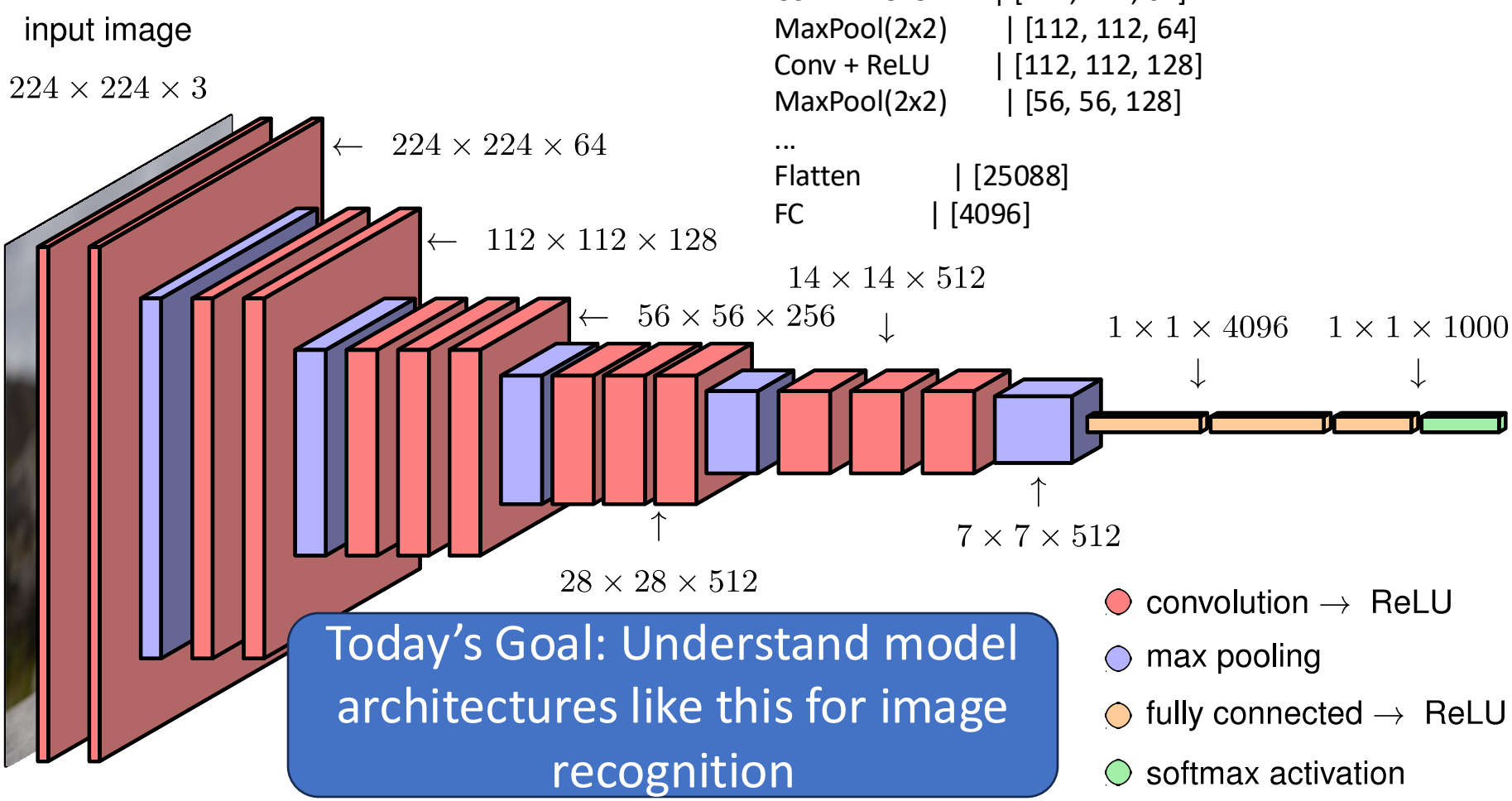
70% probability “cat”



90% probability “true” is your face

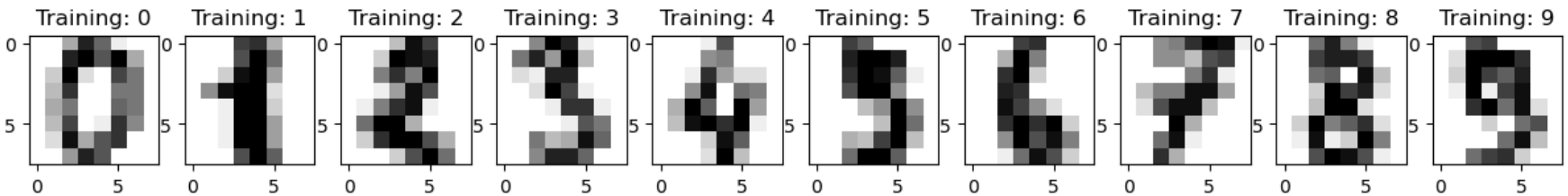
Example Image Recognition Convolutional Neural Network Architecture (VGG-16)

Layer	Shape
Input	[224, 224, 3]
Conv + ReLU	[224, 224, 64]
MaxPool(2x2)	[112, 112, 64]
Conv + ReLU	[112, 112, 128]
MaxPool(2x2)	[56, 56, 128]
…	
Flatten	[25088]
FC	[4096]



Today's Goal: Understand model architectures like this for image recognition

Reminder: Image Recognition as Classification (often with > 2 classes)



- To predict one of k classes (e.g., $0, 1, \ldots, k-1$).
- Predict the conditional probabilities:

$$\Pr(y = 0 \mid \boldsymbol{x}),\ \Pr(y = 1 \mid \boldsymbol{x}), \ldots, \Pr(y = k-1 \mid \boldsymbol{x})$$

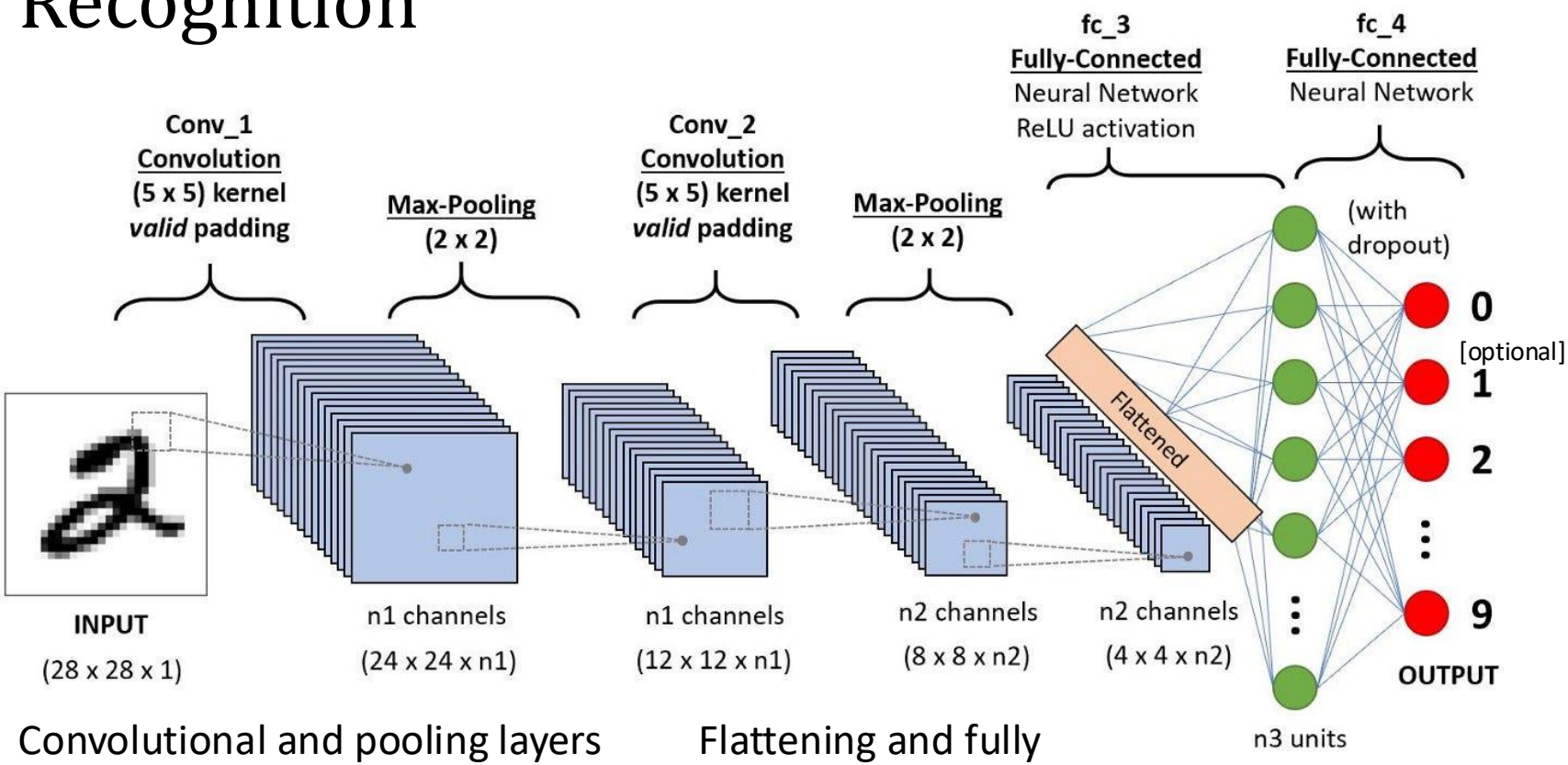
And choose the highest probability class.

Unormalized scores $\{z_i\}$ as output of last linear layer of neural network, **softmax** σ to normalize.

$$\Pr(y = i|\boldsymbol{x}) = \sigma(z_i)$$

$$= \frac{e^{z_i}}{\sum_{j=0}^{k-1} \mathrm{e}^{z_j}}$$

The Big Picture: Example Convolutional Neural Network Architecture for Image Recognition



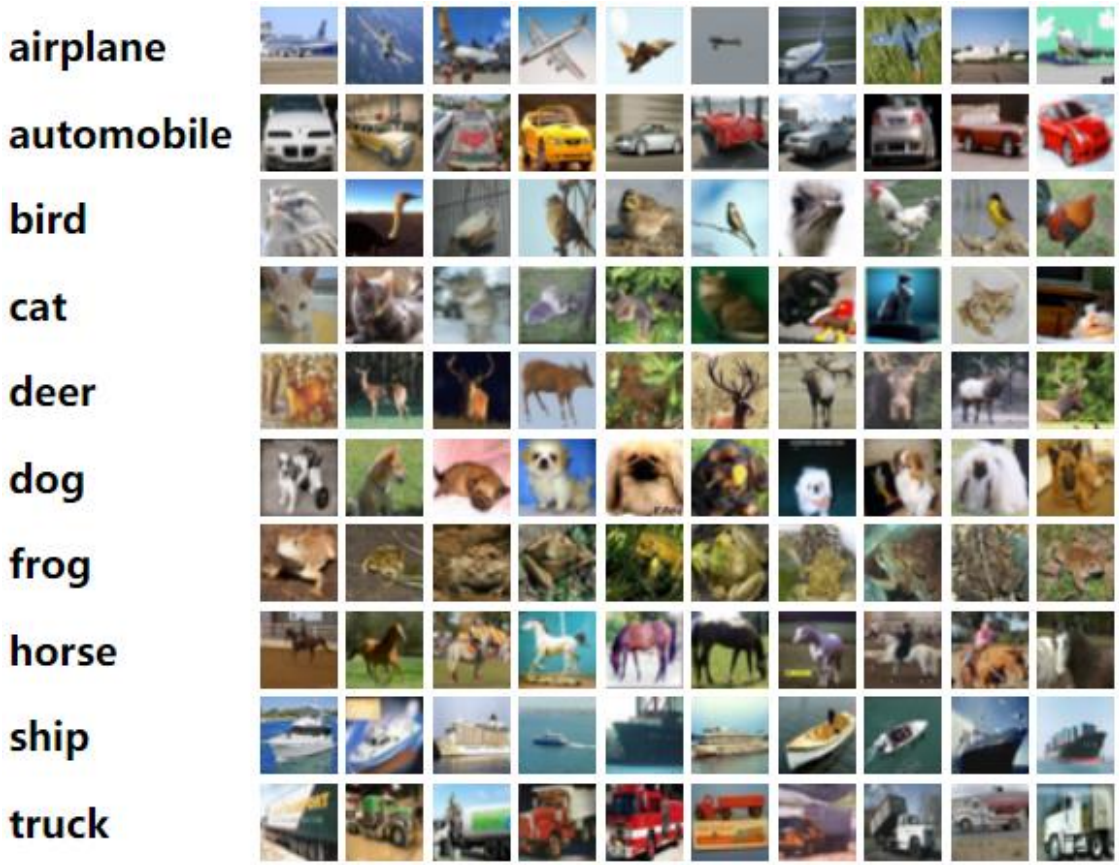
Convolutional and pooling layers learn local representations and reduce dimensionality.

Efficiently with sparse connections and parameter sharing.

Flattening and fully connected linear layer with nonlinear activation (like an MLP) to learn a global representation as a function of the local.

Final linear decision/classificatio n layer – softmax if you need to normalize.

Example Tensor for Image Data



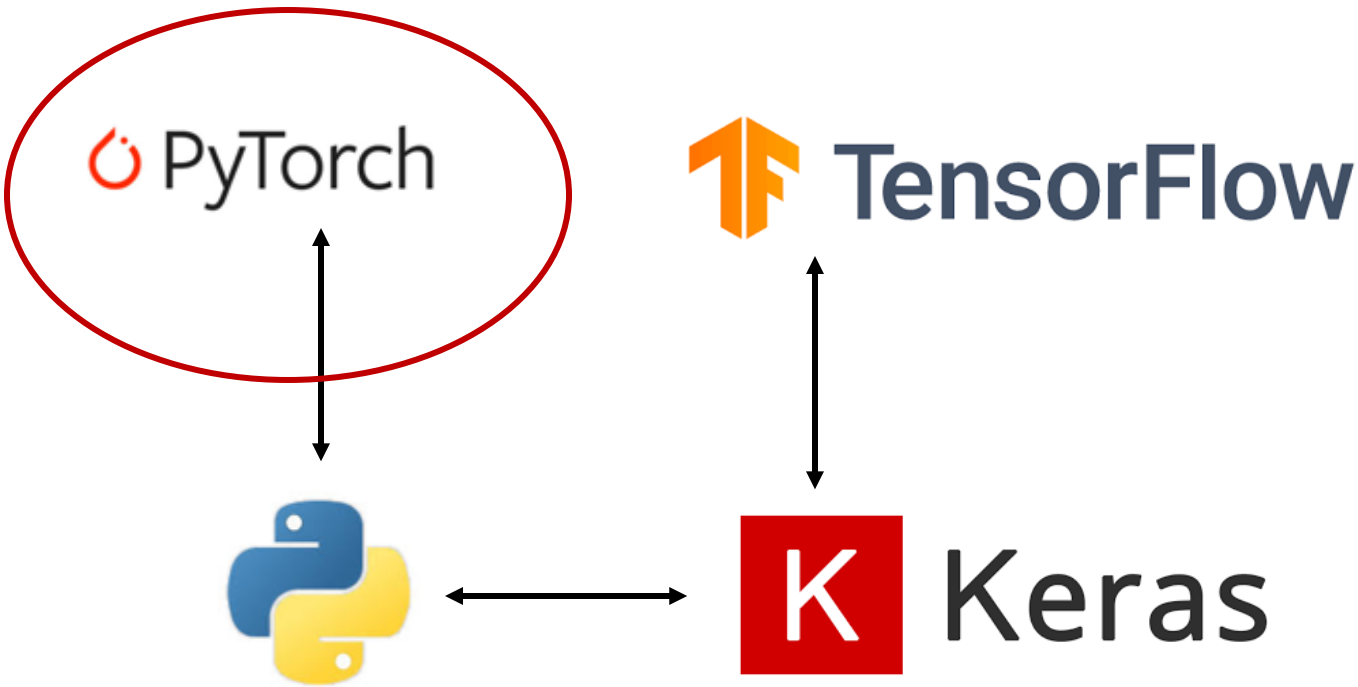
Might have a rank 4 tensor of shape: $[128, 3, 32, 32]$

- Batch of 128 images,
- 3 color channels each,
- 32 pixels in height,
- 32 pixels in width,

Total of 393,216 scalar values in total, ~393kB.

Open Source General Purpose Deep Learning Libraries

import torch



https://pytorch.org
https://www.tensorflow.org
https://keras.io

We will use PyTorch extensively in this class starting with convolutional neural networks and computer vision applications

From Matrices to Tensors

```
ar = np.array([[[1, 2], [3, 4]],
               [[5, 6], [7, 8]]])
te = torch.tensor(ar)
print(te.shape)
print(te)
```

Tensors generalize the notion of a vector or matrix along additional axes

Rank 0:

(scalar)

Rank 1:

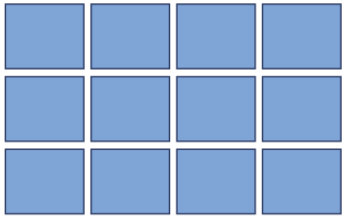
(vector)

torch.Size([2, 2, 2])

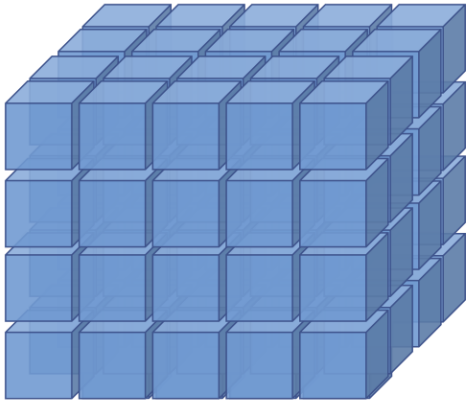
tensor([[[1, 2],
[3, 4]],

[[5, 6],
[7, 8]]])

Rank 2: (matrix)



Rank 3:



Indexing and Slicing Tensors

PyTorch tensors can be indexed and sliced just like NumPy arrays

```
tensor = torch.ones(4, 4)
print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f"Last column: {tensor[..., -1]}")
tensor[:,1] = 0
print(tensor)
```

```
First row: tensor([1., 1., 1., 1.])
First column: tensor([1., 1., 1., 1.])
Last column: tensor([1., 1., 1., 1.])
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

.item() to convert a tensor with a single value to a normal Python type:

```
print(torch.sum(tensor))
print(torch.sum(tensor).item())
```

```
tensor(12.)
12.0
```

Vectorized Operations on Tensors

A [2,2,2] shape rank 3 tensor

```
te

tensor([[[1, 2],
         [3, 4]],

        [[5, 6],
         [7, 8]]])
```

Applied element-wise

```
te * 2 + 10

tensor([[[12, 14],
          [16, 18]],

         [[20, 22],
          [24, 26]]])
```

Matrix multiplication broadcast across first dimension

```
te @ te

tensor([[[  7, 10],
         [ 15, 22]],

        [[ 67, 78],
         [ 91, 106]]])
```

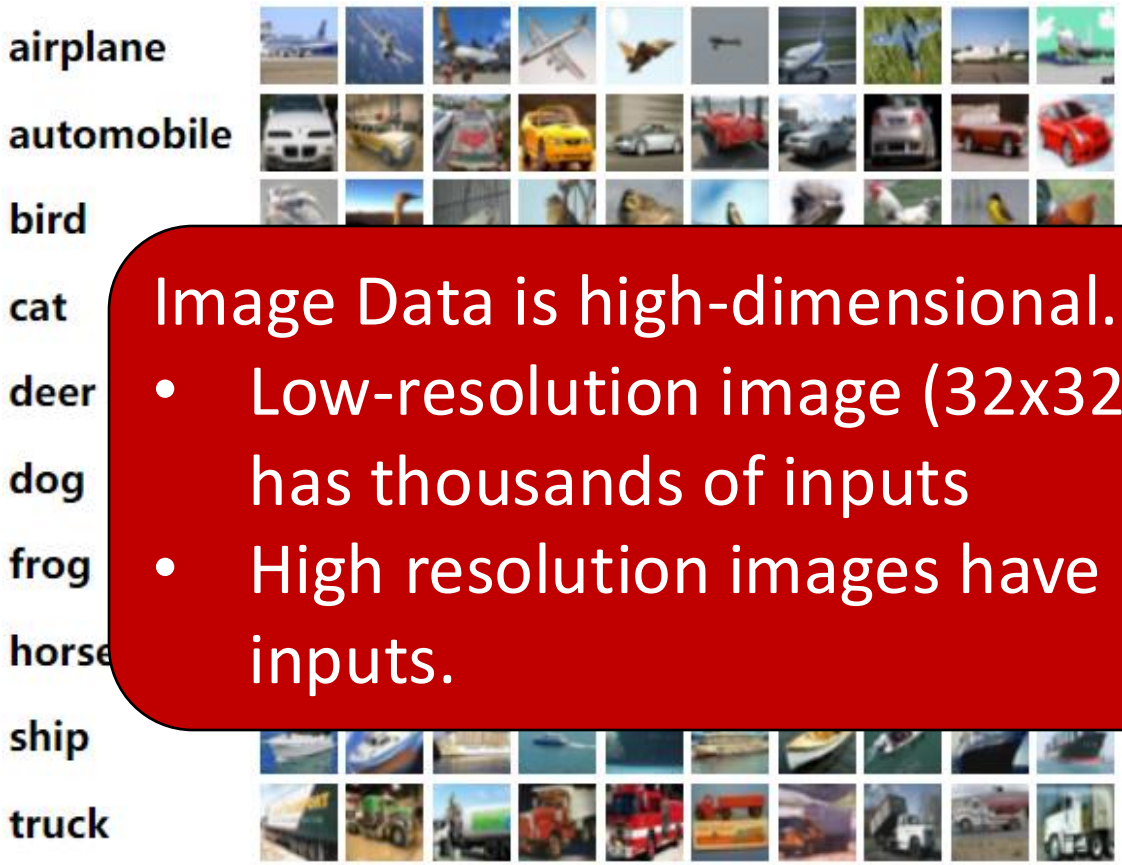
```
te[0] @ te[0]

tensor([[ 7, 10],
        [15, 22]])

te[1] @ te[1]

tensor([[ 67,  78],
        [ 91, 106]])
```

Example Tensor for Image Data



Might have a rank 4 tensor of shape: $[128, 3, 32, 32]$

Image Data is high-dimensional.

- Low-resolution image (32x32 pixel in color) has thousands of inputs
- High resolution images have millions of inputs.

Multilayer Perceptron How Many Parameters?

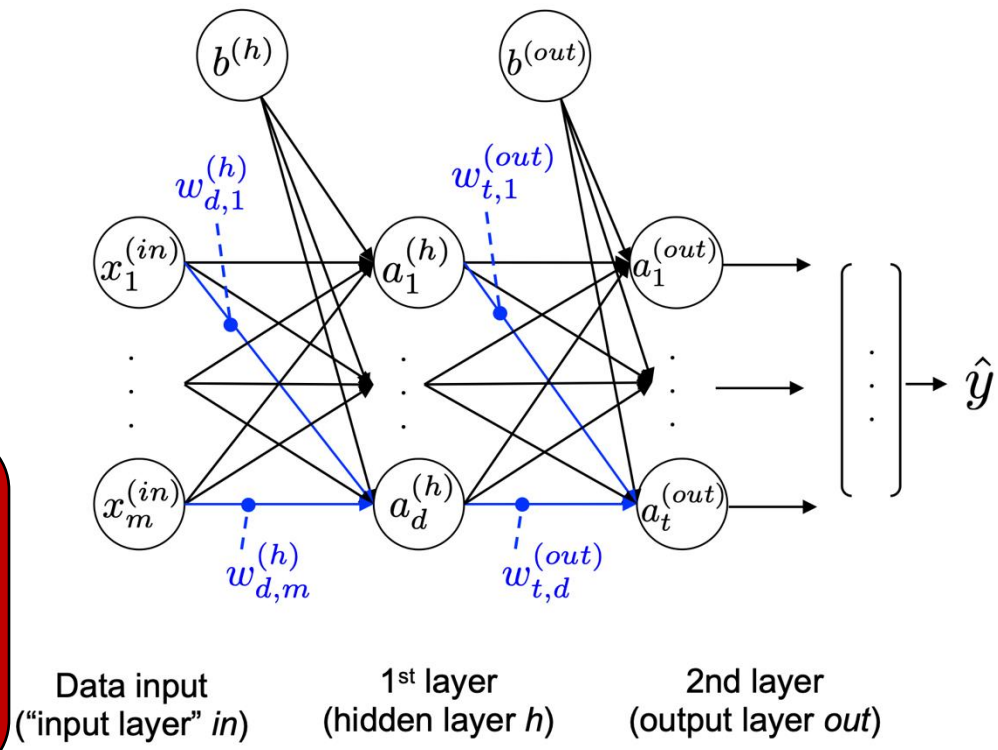
$$\boldsymbol{a}^{(h)} = W^{(h)}\,\boldsymbol{x}^{(in)} + b^{(h)}$$

$$\boldsymbol{a}^{(out)} = W^{(out)}\,\sigma\left(\boldsymbol{a}^{(h)}\right) + b^{(out)}$$

Multiplies weight matrix by inputs, add bias, then element-wise activation.

MLP with 32x32 images and 1024 hidden units would need > 1 million parameters.

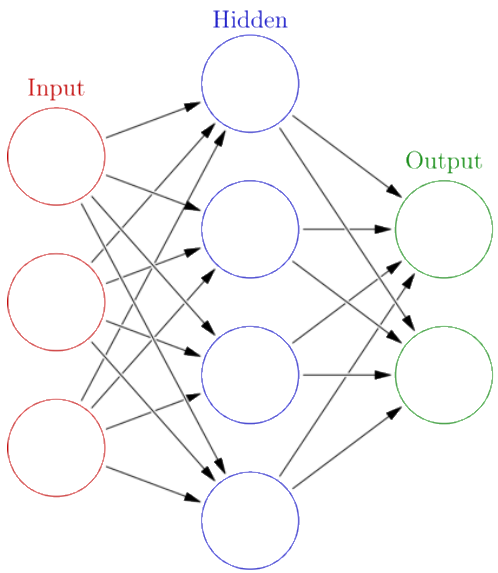
MLP with 1,000x1,000 images and 10,000 hidden units would need > 10 billion parameters!



Big Question:
How can we use fewer parameters to high dimensional image data?

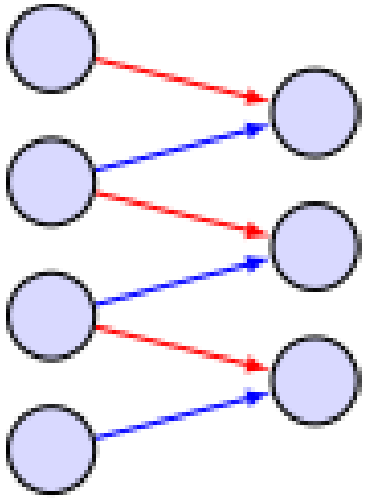
Sparsity and Parameter Sharing

Instead of a a ***fully connected*** linear layer with ***independent parameters***.



In the fully connected linear layers of the MLP architecture, all units are connected, and every connection has an independent model parameter.

Could use ***sparse connections*** with ***shared parameters***.



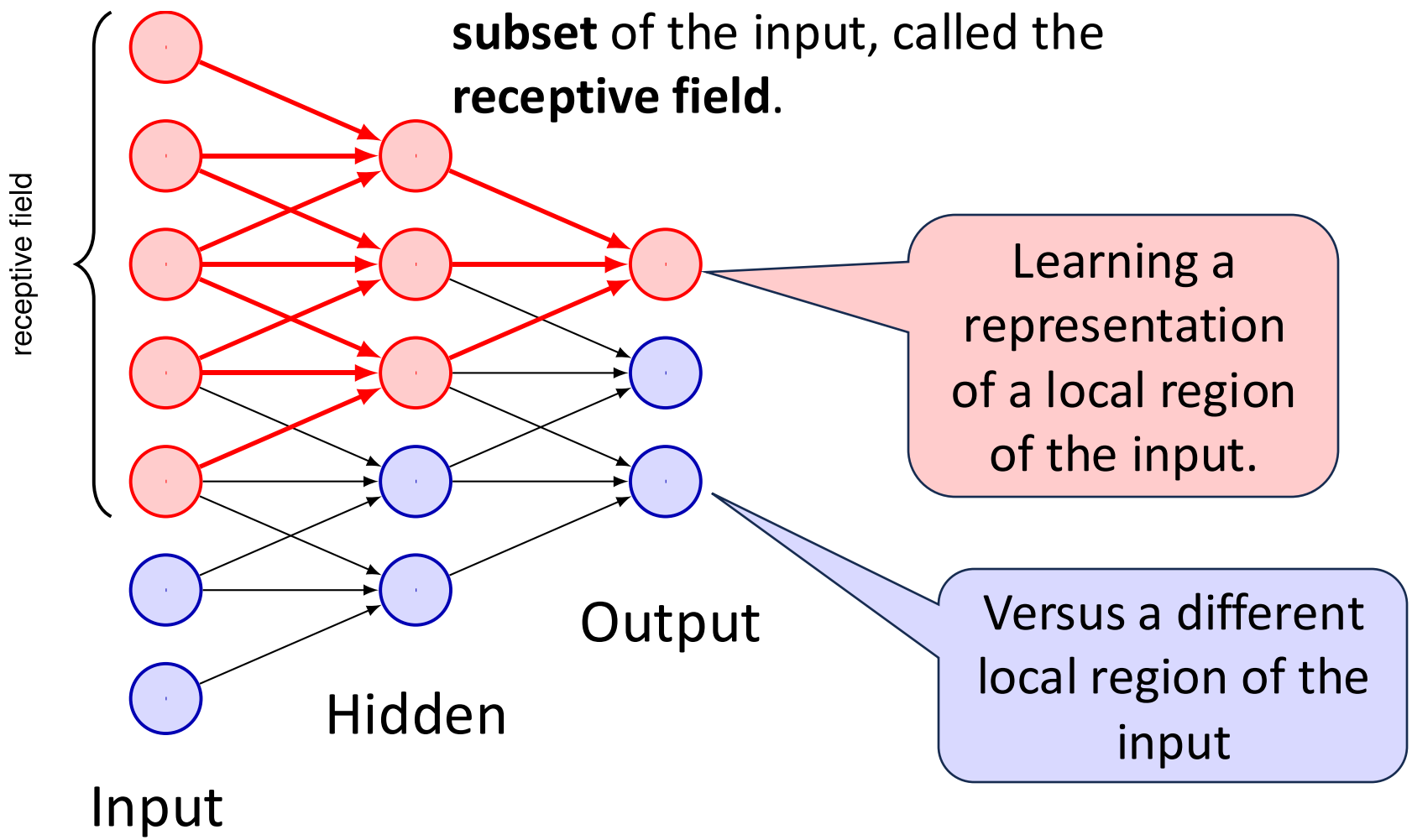
Parameter 1
Parameter 2

parameters constant with respect to # input and # output units

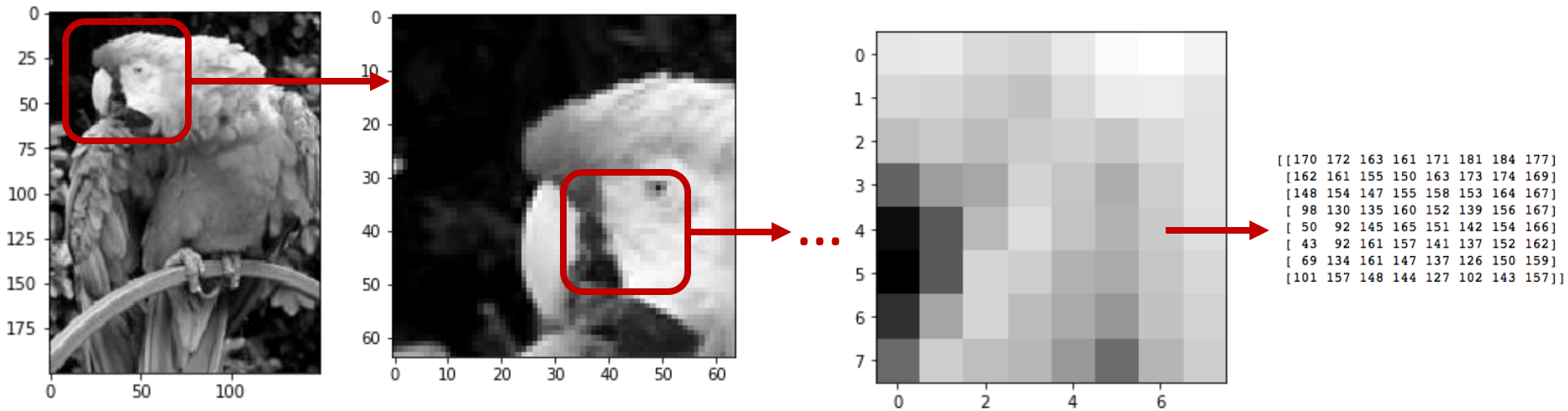
In convolutional layers, only some units are connected and the same parameter may be shared across multiple connections.

The Receptive Field in a Sparse Network

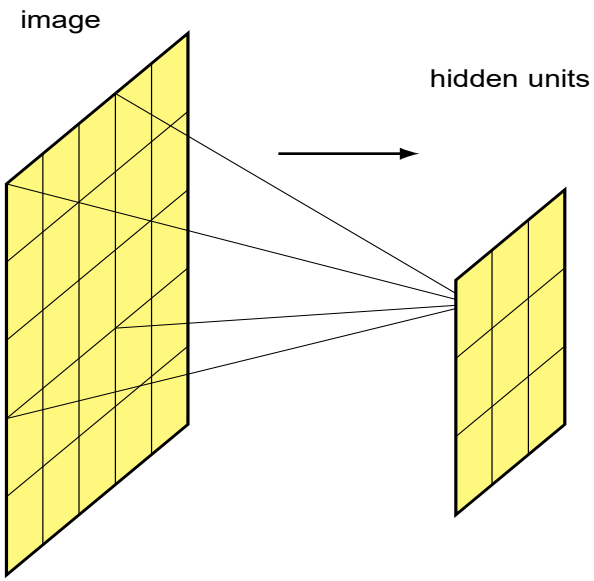
For successive convolutional layers, a given unit is a function of **only a subset** of the input, called the **receptive field**.



Images, Computer Vision, Sparsity and the Receptive Field



Beyond the flattened 1D representation, use sparse 2D convolutional layers to learn local representations of image regions.



Example of Representations A 2D Convolution Can Learn

Example 2D Convolutional Kernels

Vertical Edges

-1	0	1
-1	0	1
-1	0	1

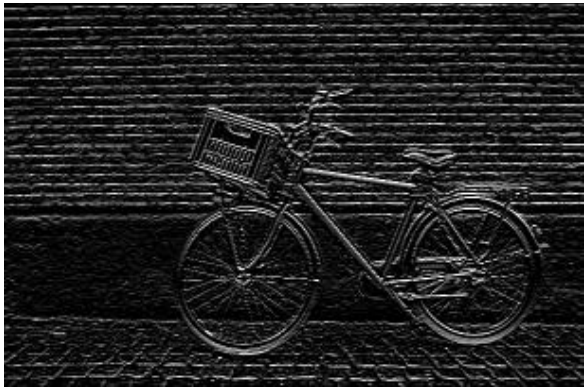
Horizontal Edges

-1	-1	-1
0	0	0
1	1	1

Original Image

(a)

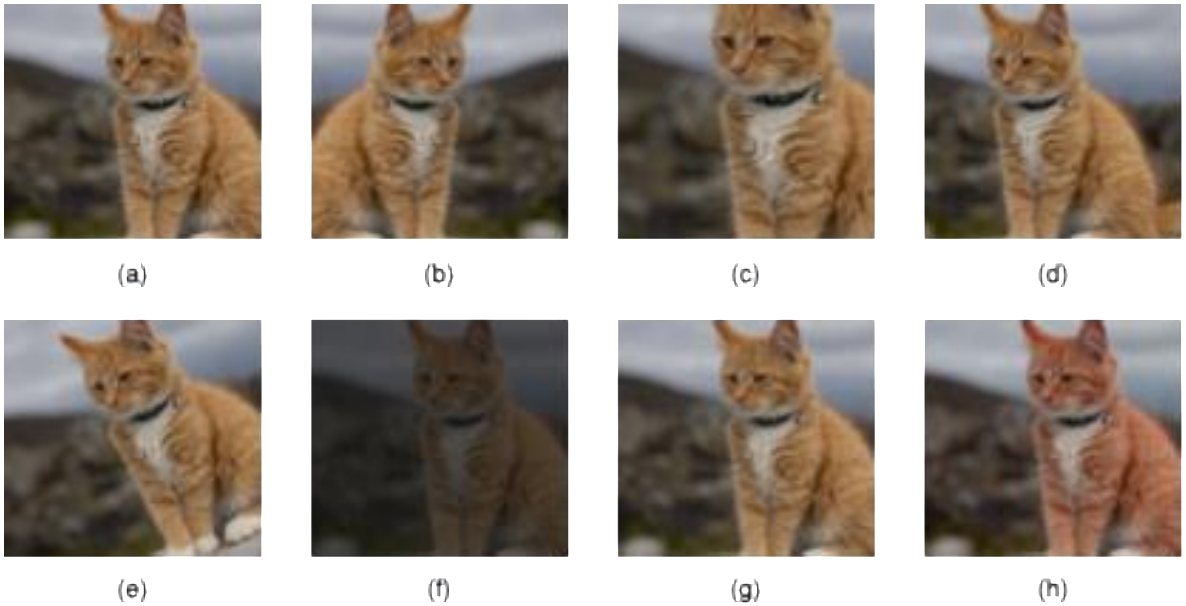
(b)



(c)

Why Parameter Sharing Makes Sense: Translation Invariance

Translation Invariance: We should be able to detect the cat's face in all these images, even though it appears in different places in different images.



Data Augmentation – For the same reason, in computer vision it is easy to *augment your training dataset* by performing flips, crops, and recolorings like these to make model training more robust.

Two-Dimensional Convolution

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

These are the parameters we learn.

320					

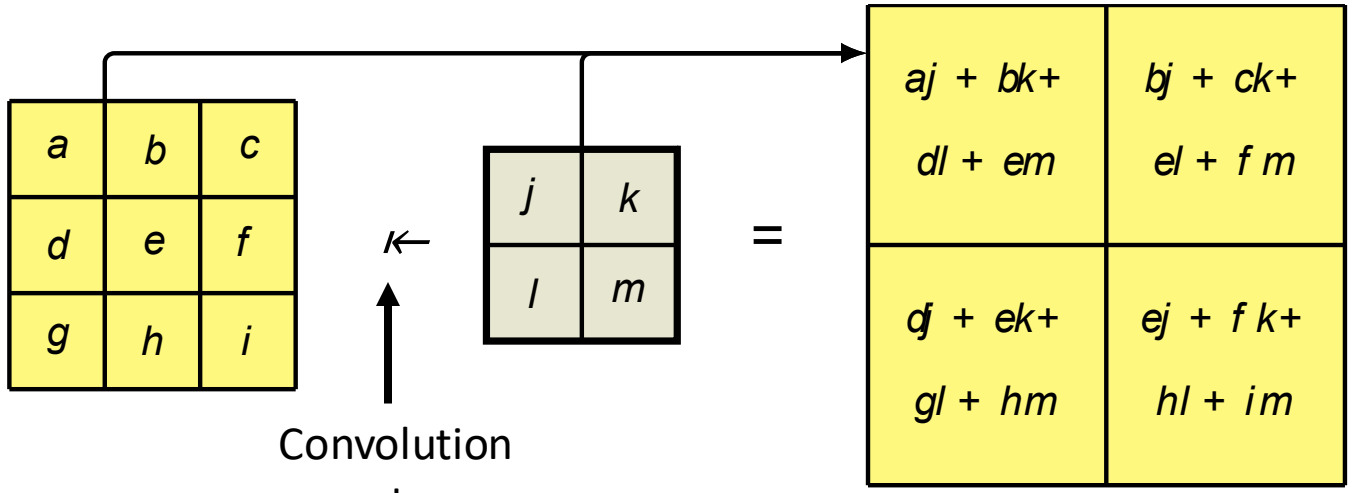
Output Matrix

$$0*0+0*-1+0*0$$
$$+0*-1+105*5+102*-1$$
$$+0*0+103*-1+99*0 \quad = 320$$

Convolution with horizontal and vertical strides = 1

Aside: Technically this is a cross-correlation, not a mathematical convolution (which are not commonly used in applied ML). We will stick with the convolution and refer to them as convolutions.

Two-Dimensional Convolution



Convolution operator

Input I | Kernel K | Pre-Activation Results C

Or "filter." **These are the parameters we learn.** Bias term not shown.

To be followed by elementwise nonlinear activation (e.g., ReLU)

Suppose:

- Input I has dimensions W by H
- Kernel K has dimensions F by F

Then the convolved results are:

- $C_W = W - F + 1$ by
- $C_H = H - F + 1$

$$C(x, y) = \sum_{s=0}^{F-1} \sum_{t=0}^{F-1} I(x + s, y + t) K(s, t)$$

(assuming a stride of 1 and no padding)

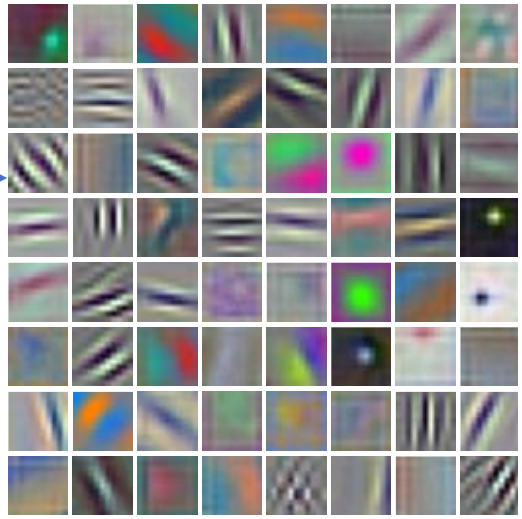
Optional: Padding Convolutions

Note how the size of the feature map *shrinks* after a convolution. If undesired, can *pad* so that the output has the same shape.

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

Visualizing Convolutional Net Representations

Visualization of various learned convolutional filters in first layer.



Validation images with highest activations by filter at different layers. Increasing abstraction.

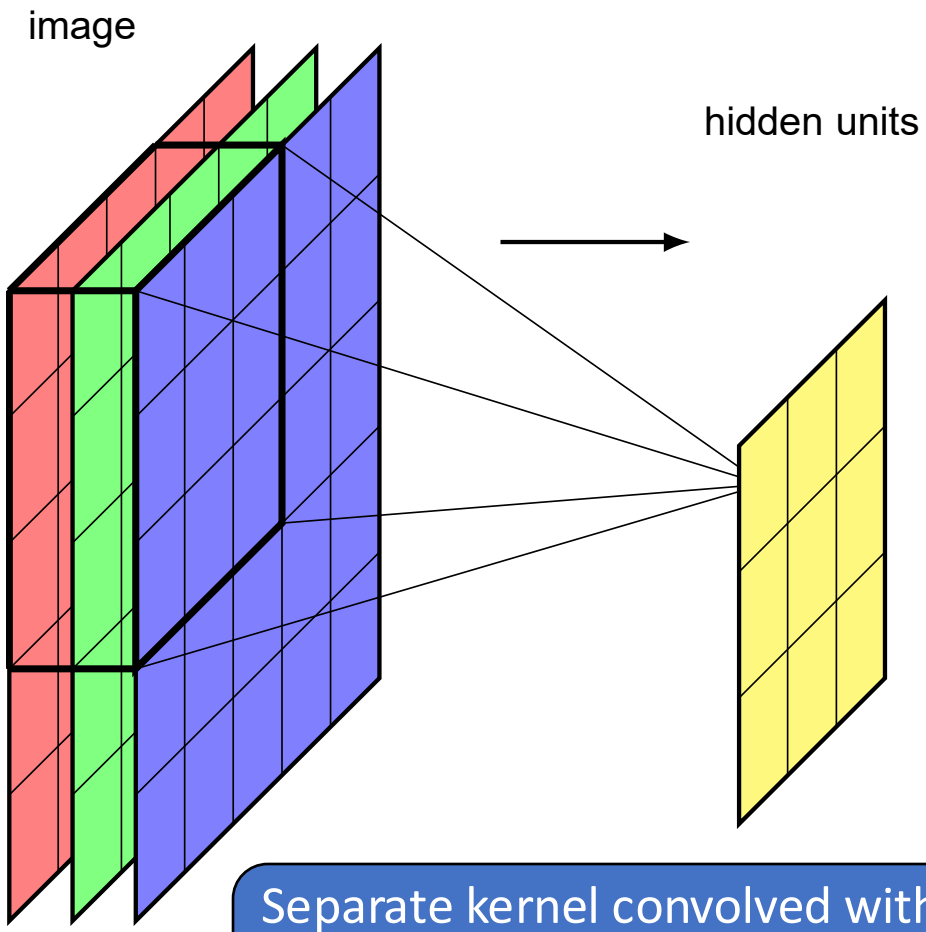
Layer 1

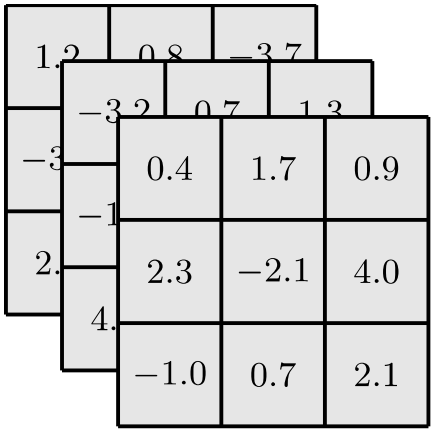
Layer 2

Layer 3

Layer 5

Multiple Channels In





For n input channels $I_1, I_2, \ldots, I_n$, kernels $K_1, K_2, \ldots, K_n$, and bias b…

$$C_{\text{out}} = b + \sum_{j=1}^{n} I_j * K_j$$

Separate kernel convolved with each input, summed to value of hidden unit. Optional extra bias term.

Convolution with Multiple In Channels

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2

0	1	1
0	1	0
1	-1	1

Kernel Channel #3

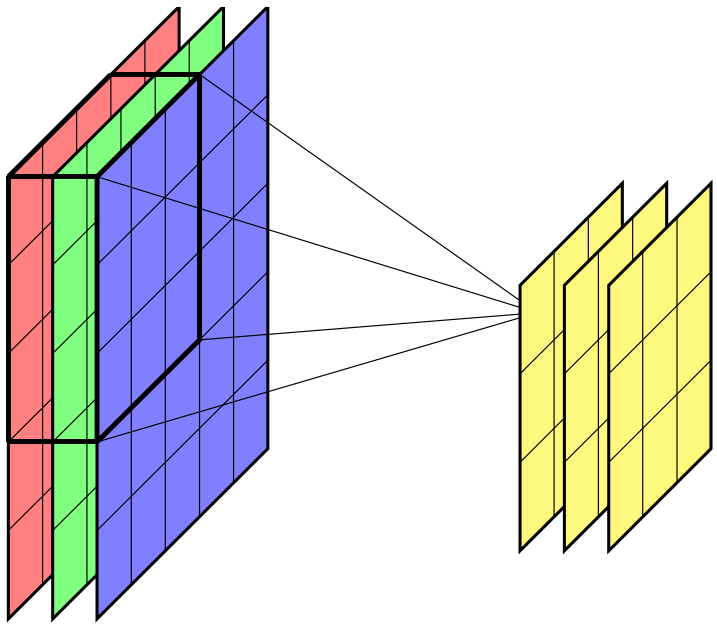
$$308 + -498 + 164 + 1 = -25$$

Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

Multiple Out Channels



For n input channels $I_1, I_2, \dots, I_n$, and m output channels $C_1, C_2, \dots, C_m$,

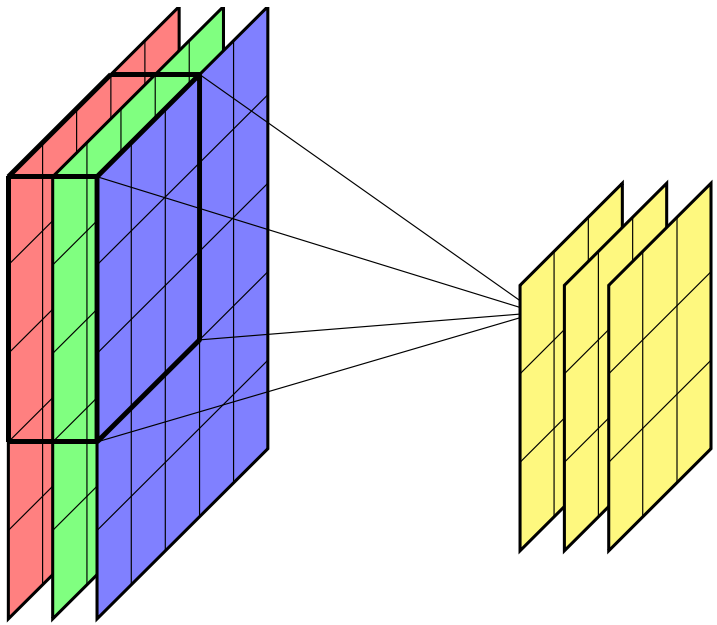
We have nm kernels
$K_{1,1}, K_{2,1}, \dots, K_{n,1}$,
$K_{1,2}, K_{2,2}, \dots, K_{n,2}$,
…
$K_{1,m}, K_{2,m}, \dots, K_{n,m}$

And m biases $b_1, b_2, \dots, b_m$

$$C_{\mathrm{i}} = b_i + \sum_{j=1}^{n} I_j * K_{j,i}$$

Separate parameters / kernel for each out channel.
Primary way to increases model capacity of convolutional layers.

Multiple Out Channels



For n input channels $I_1, I_2, \dots, I_n$, and m output channels $C_1, C_2, \dots, C_m$,

We have nm kernels

$K_{1,1}, K_{2,1}, \dots, K_{n,1}$,

$K_{1,2}, K_{2,2}, \dots, K_{n,2}$,

…

$K_{1,m}, K_{2,m}, \dots, K_{n,m}$

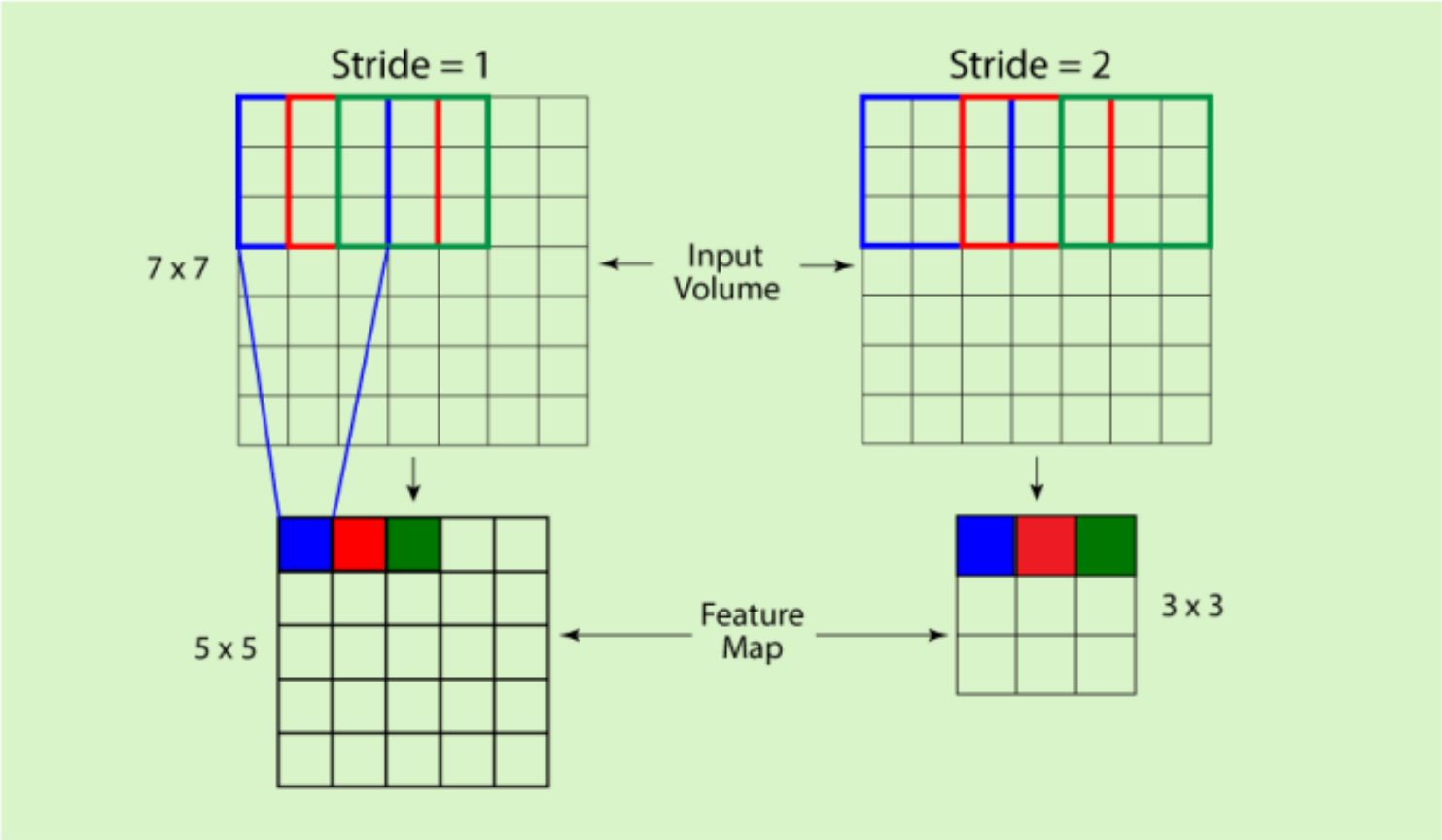
For $k \times k$ kernels, a total of $(k^2 n + 1)m$ learnable parameters

Example: 4×4 kernel, 3 in channels and 10 out channels: $(4^2 \cdot 3 + 1) \cdot 10 = 490$ learnable parameters

Crucially: The number of parameters DOES NOT DEPEND on the resolution of the image!

$j=1$

Alternate Choices of Stride

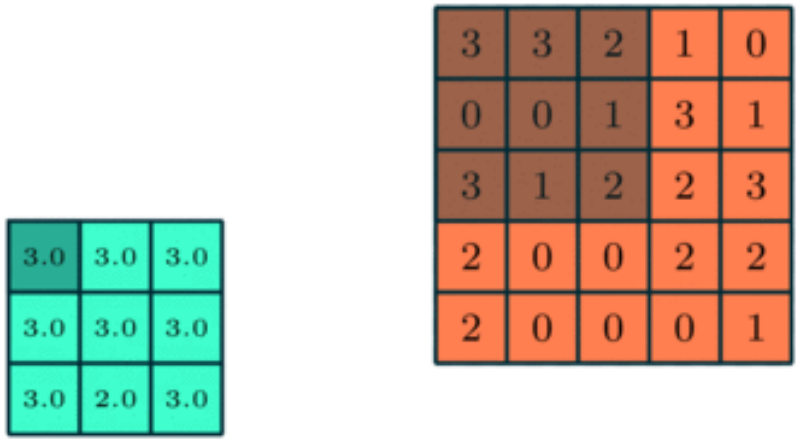


Increasing stride results in small output feature maps, compressing the input.

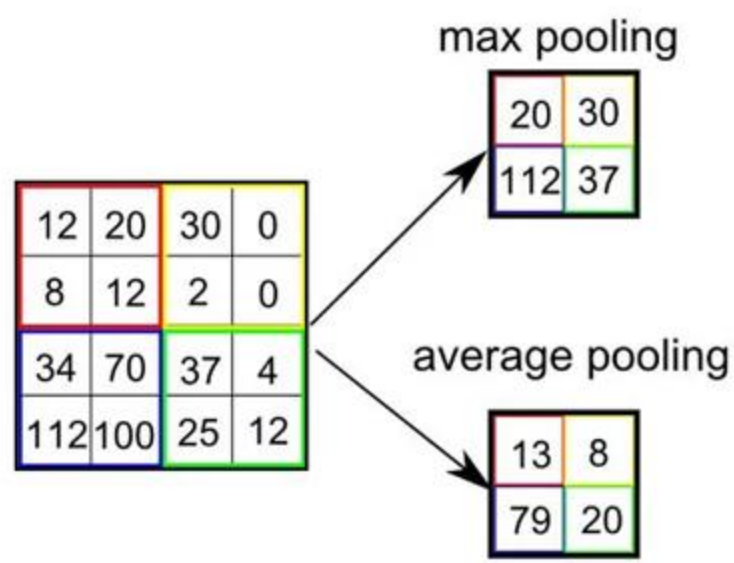
Pooling

Pooling has no learnable parameters but reduces the dimensionality and smooths noise.

Commonly used after a convolution layer.



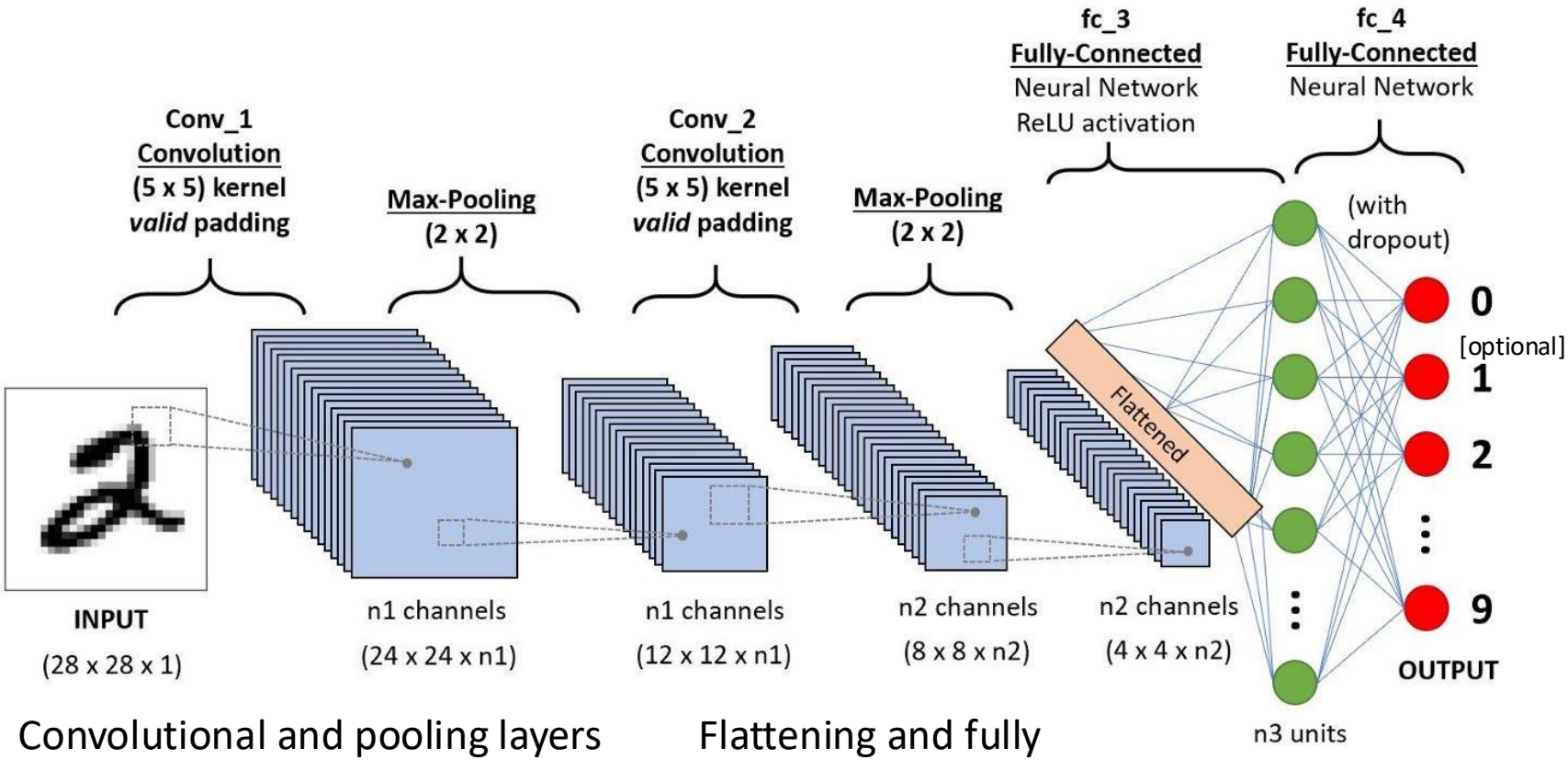
Pooling with stride 2



Types of Pooling

Max Pooling is most common in practice.

Example Convolutional Neural Network Architecture for Image Recognition



Convolutional and pooling layers learn local representations and reduce dimensionality.

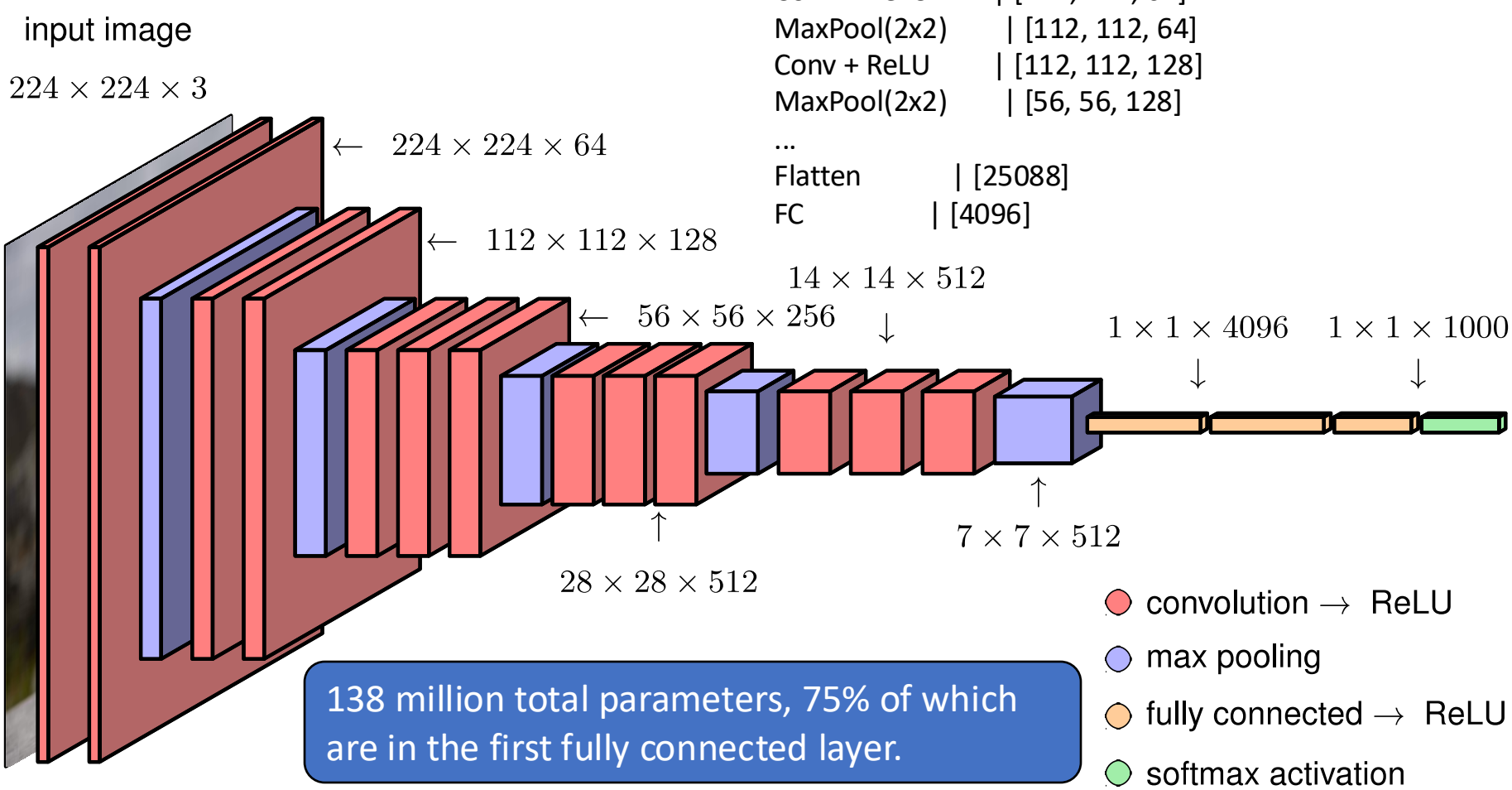
Efficiently with sparse connections and parameter sharing.

Flattening and fully connected linear layer with nonlinear activation (like an MLP) to learn a global representation as a function of the local.

Final linear decision/classificatio n layer – softmax if you need to normalize.

Example Image Recognition Convolutional Neural Network Architecture (VGG-16)

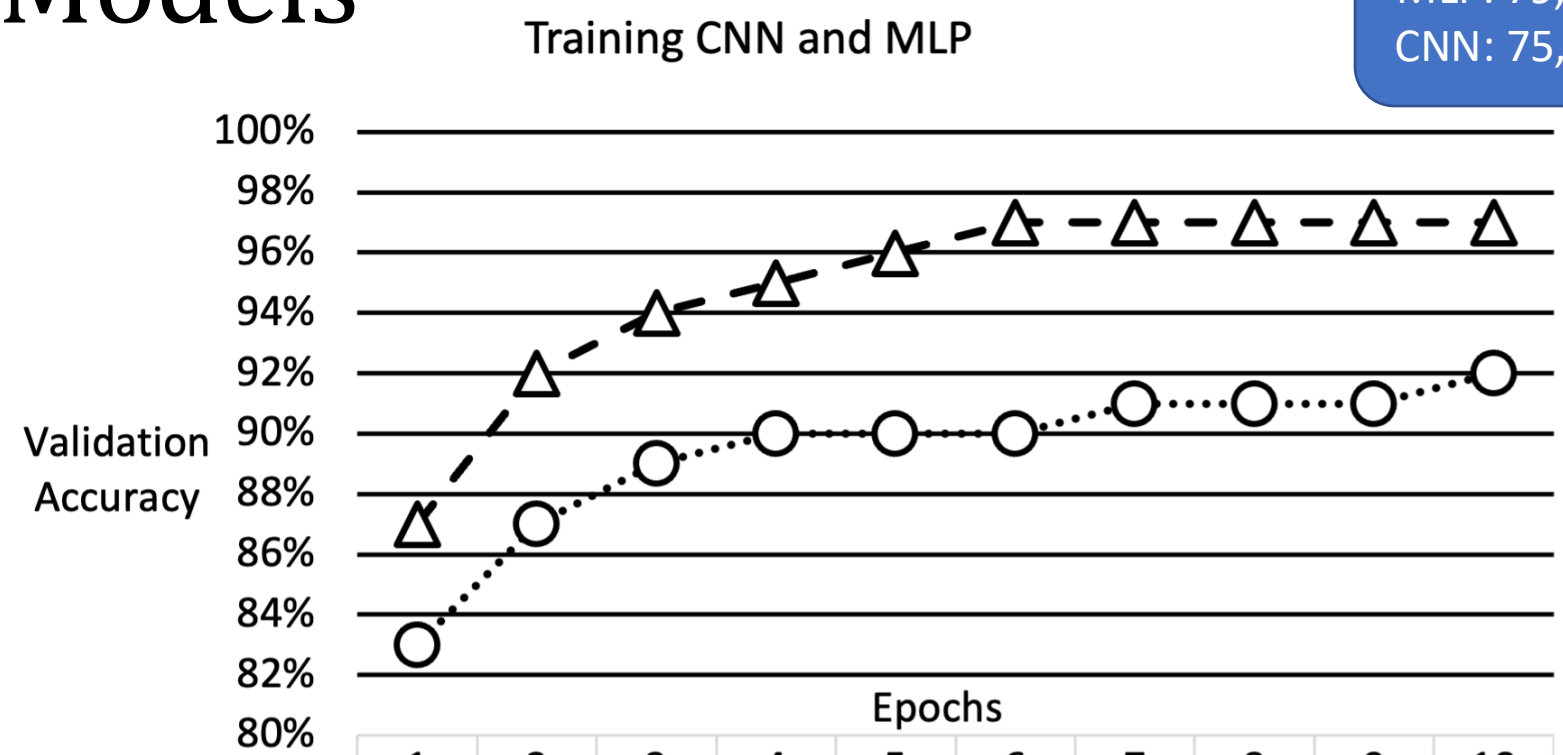
Layer	Shape
Input	[224, 224, 3]
Conv + ReLU	[224, 224, 64]
MaxPool(2x2)	[112, 112, 64]
Conv + ReLU	[112, 112, 128]
MaxPool(2x2)	[56, 56, 128]
...	
Flatten	[25088]
FC	[4096]



138 million total parameters, 75% of which are in the first fully connected layer.

Comparing MLP and CNN Models

Model sizes:
MLP: 79,510
CNN: 75,430



	1	2	3	4	5	6	7	8	9	10
MLP	83%	87%	89%	90%	90%	90%	91%	91%	91%	92%
CNN	87%	92%	94%	95%	96%	97%	97%	97%	97%	97%

Convolutional neural networks often outperform MLPs of similar size on computer vision tasks.